

A Digital Twin Framework for Traffic-Aware UAV Pavement Monitoring in Open-Traffic Conditions

Yamil Uchani¹ , Grace Luna^{1,‡} , Edwin Salcedo^{2,‡,*} , Mauricio Figueroa¹ 

¹*Department of Mechatronics Engineering, Universidad Católica Boliviana “San Pablo”, Bolivia*

²*School of Electronic Engineering and Computer Science, Queen Mary University of London, United Kingdom*

[‡]These authors contributed equally to this work.

*Corresponding author: e.r.salcedoaliaga@qmul.ac.uk

Abstract—UAV-based pavement inspection can reduce the cost and risk of road-surface monitoring, but real-world deployment remains difficult when traffic, pedestrians, and temporary occlusions affect defect visibility. This paper presents a Unity-based digital twin framework for traffic-aware UAV pavement monitoring in open-traffic conditions. The proposed environment integrates procedurally generated road defects, dynamic traffic agents, autonomous UAV navigation, and a multitask YOLOv8n perception module for detecting road defects, pedestrians, and vehicles while classifying road-defect subtypes. After synthetic-domain fine-tuning, the perception model achieved 0.959 mAP@0.5 and 0.940 macro F1-score on a held-out synthetic test set generated from the simulator. The digital twin was then used to evaluate hover-and-recheck, micro-repositioning, and skip-and-revisit recovery strategies across different traffic densities and flight altitudes. Results show that flight altitude strongly affects inspection coverage, while recovery strategies introduce different trade-offs between coverage, mission duration, energy consumption, and revisit behaviour. These findings demonstrate that digital twins can support the development and evaluation of traffic-aware UAV inspection strategies before real-world deployment. The full implementation and trained models are available at <https://github.com/EdwinTSalcedo/RDMO-DigitalTwin>.

Index Terms—Pavement Monitoring, Digital Twin, Traffic-Aware UAV Inspection, Recovery Strategies

I. INTRODUCTION

Road infrastructure is vital to mobility, economic development, and public safety, yet poor road conditions remain a major cause of accidents, vehicle damage, and costly maintenance worldwide. Road-surface defects such as potholes, cracks, and asphalt delamination can accelerate vehicle wear, degrade ride quality, and create hazardous driving conditions under adverse weather, low-visibility, or high-traffic conditions. For this reason, timely and reliable inspection is essential for identifying road defects and supporting maintenance decisions. Current road inspection approaches rely on manual visual assessment or computer-vision-based inspection pipelines [1], [2]. More recently, the advent of unmanned aerial vehicles (UAVs) and edge computing has enabled greater automation in road inspection tasks [3]–[5].

However, despite these advances, existing UAV-based inspection methods remain subject to operational limitations, human error, and safety risks, especially when monitored areas are large or experience high traffic volumes. In such dynamic environments, inspections may require temporary traffic restrictions to ensure road-surface visibility, causing

disruption and economic losses. Moreover, road-inspection computer-vision pipelines validated under controlled conditions may perform less reliably when exposed to moving road users and partially occluded defects. Simulation and digital-twin platforms can help reduce this gap by enabling safer, repeatable, and traffic-aware evaluation of UAV inspection strategies before real-world deployment.

Although prior work has explored maintenance-oriented digital twins [6]–[9] and UAV inspection-route planning [4], [10], existing approaches rarely provide a unified traffic-aware testbed that integrates realistic road-surface defect generation, dynamic vehicle and pedestrian flows, UAV inspection behaviour, and recovery strategies for temporarily occluded road regions. This limits the development and validation of UAV-based inspection systems under realistic operating conditions. To address this gap, we present a Unity-based digital-twin framework for UAV-driven road-damage inspection. The framework combines procedural road-defect generation with simulated vehicle and pedestrian traffic, enabling the evaluation of inspection strategies when road regions are temporarily obstructed. Rather than assuming that recovery actions always improve inspection performance, the framework enables evaluation of recovery strategies and reveals their trade-offs in terms of coverage, mission duration, energy consumption, and revisit behaviour. Experiments demonstrate its potential for assessing traffic-aware UAV inspection before deployment.

II. RELATED WORK

UAV-based inspection has become a practical approach for rapid and high-resolution pavement monitoring. Recent studies have shown that UAV imagery, photogrammetry, and deep learning (DL) can support automated detection and assessment of road-surface defects. For example, Aburqaq et al. [11] demonstrated the use of UAV-based photogrammetry for flexible pavement inspection, while Alzamzami et al. [12] proposed PDS-UAV, a YOLOv8-based pothole detection system using UAV imagery. Large-scale datasets and benchmark studies, such as RDD2022 [13], have further supported the development of DL models for road-damage detection across different countries, defect types, and environmental conditions. In parallel, lightweight models such as YOLOv8-PD [14], YOLO9tr [15], and MobiLiteNet [16] have demonstrated the feasibility of accurate and deployable road-distress monitoring.



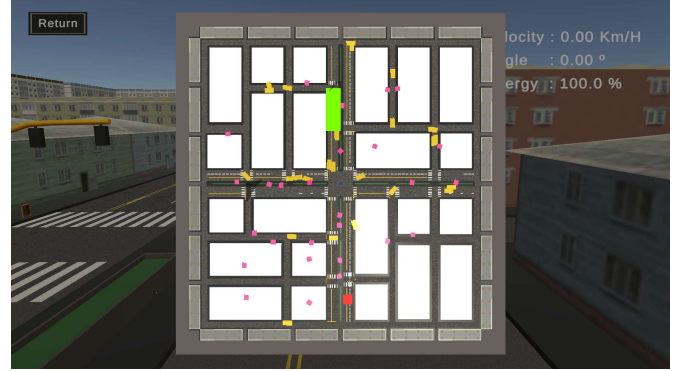
(a)



(b)



(c)



(d)

Fig. 1: Main components of the digital twin simulation framework: (a) ground vehicle, (b) unmanned aerial vehicle (UAV), (c) inspection perspective from the UAV, and (d) map view. The green region denotes a road segment selected for inspection. The red dot represents the UAV, while the yellow and pink dots represent cars and pedestrians, respectively.

Simulation environments and digital twins (DTs) have been increasingly explored to reduce dependence on field data collection and enable controlled evaluation of road-inspection systems. Tsai et al. [17] generated virtual pothole scenes under different surface conditions and combined synthetic with real data to enhance road-distress detection. Wang et al. [18] reconstructed UAV-derived pavement backgrounds and rendered multiple distress classes under challenging visual conditions, while Wang et al. [19] integrated textured pavement-background modelling with Unreal Engine-based virtual UAV inspection to improve detection performance through synthetic-data augmentation. Collectively, these studies highlight the value of simulation for road-distress data generation and detector training.

System-level UAV inspection and digital-twin frameworks have also been proposed for road monitoring, route planning, and maintenance support. Silva et al. [3] introduced a distributed multi-agent architecture for pothole recognition in UAV images, Zhao et al. [4] combined UAV path planning with image splicing for pavement-damage detection, and Zhong et al. [10] investigated UAV swarm route planning for road inspection. In infrastructure management, DTs have been used for pavement-health monitoring, predictive maintenance, and decision support [6]–[9]. However, existing system-level,

route-planning, and digital-twin approaches remain largely focused on image acquisition, monitoring architectures, route optimisation, or asset-level maintenance. Fewer studies provide a unified traffic-aware simulation testbed that jointly evaluates UAV inspection missions, procedural road-defect generation, dynamic vehicle and pedestrian occlusions, in-flight detection, tracking, and recovery strategies before real-world deployment.

III. DIGITAL TWIN SIMULATION FRAMEWORK

This work proposes a digital twin (DT) simulator designed to provide a controlled and reproducible environment for UAV-based pavement-inspection research. It represents a configurable urban road network and supports experimentation with computer vision and machine learning (ML) algorithms. The simulator enables data acquisition through two inspection platforms: a ground vehicle and a UAV, as shown in Figure 1. The following sections describe its main components.

A. Unity-Based Environment

The virtual environment was developed in Unity, with selected assets modelled in Blender, to represent an urban road network composed of streets with varying widths, intersections, traffic lights, and road signs. Within this environment, the simulator includes a ground vehicle based on

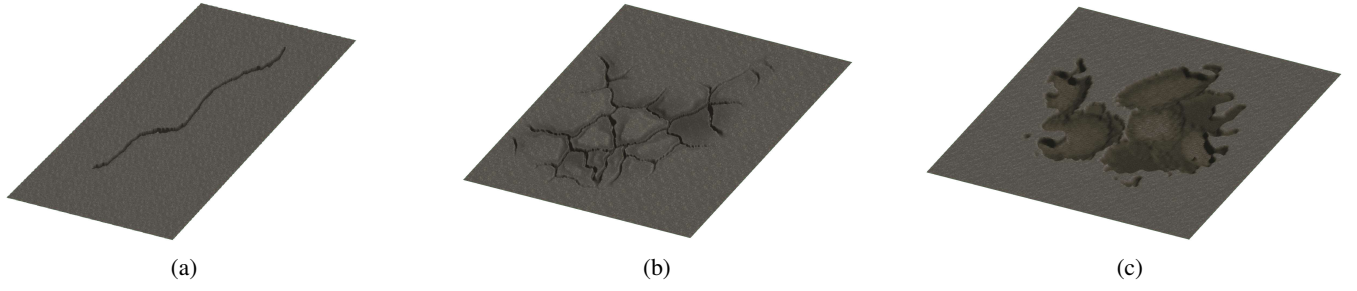


Fig. 2: Isometric views of procedurally generated road-surface defects: (a) single crack, (b) crocodile crack, and (c) pothole.

a 2018 Hyundai Grand i10 model and a UAV based on the DJI Phantom 4. The ground vehicle supports auxiliary data collection, although the experiments in this work focus on UAV-based inspection. To approximate realistic inspection conditions, vehicular traffic and pedestrian movement were implemented in Unity, allowing the UAV and ground vehicle to operate under dynamic urban conditions. A map of the environment with indicators for mobile elements is presented in Figure 1d.

Following the taxonomy used in [2], road-surface deterioration can be described through three widely recognised defect categories. Cracks correspond to linear pavement fractures, while interconnected fracture patterns associated with advanced deterioration are represented as crocodile cracks. Potholes, in turn, represent localised bowl-shaped depressions caused by pavement material loss. Together, these categories represent increasing levels of road-surface deterioration. These three categories were procedurally generated and integrated into the virtual environment, as shown in Figure 2. The generated defects vary in shape, size, and edge irregularity to support the assessment of different road-damage conditions within the simulated inspection environment.

B. NavMesh-Based UAV Route Planning and Inspection

The UAV operates autonomously using Unity's NavMesh navigation system. In the simulator, an inspection target is represented as a set of road segments, where each segment s is defined by start and end positions. Targets may be assigned interactively through the minimap interface or automatically by the experiment automator, which selects a randomised sequence of unvisited segments for repeatable batch evaluation. Once a target is assigned, the UAV computes a route from its position to the target segment using the baked NavMesh, which represents the traversable flight domain above the road network as a set of connected convex polygons. The navigation and inspection process consists of four stages: NavMesh generation through geometry baking, global pathfinding using Unity's internal navigation search, trajectory smoothing before execution by the *NavMeshAgent*, and segment-level inspection using the selected navigation policy.

For reproducibility, the NavMesh configuration was defined using the UAV agent dimensions and motion constraints. The agent radius was set to 0.5 m, while the inspection altitude was configurable through the simulator interface and evaluated

at three levels: 6 m, 10 m, and 15 m. The *NavMeshAgent* was configured with a speed of 5 m/s, an angular speed of $45^\circ/\text{s}$, an acceleration of 8 m/s^2 , and a stopping distance of 1 m. These parameters influence route feasibility, turning behaviour, mission time, and safe clearance during inspection. Dynamic avoidance was handled through the *NavMeshAgent* local-avoidance settings, allowing the UAV to react to nearby agents or obstacles while following the global path computed by Unity's navigation system.

After reaching the assigned road segment, the UAV follows the configured inspection pattern while the DT monitors visibility, coverage, and recovery events. When an occlusion occurs, the UAV applies the recovery strategy defined in Section III-C. Road-defect outputs from the perception module described in Section IV trigger automatic image capture and georeferencing, with samples saved only when defects are sufficiently centred in the UAV camera view. The simulator also records coverage, recovery ratio, mission time, and energy consumption for each segment, enabling the recovery strategies to be compared across traffic densities and flight altitudes. Once a segment has been processed, the UAV either proceeds to the next assigned segment or returns to its take-off position at the end of the mission.

C. Digital Twin Definition

In this work, the term digital twin refers to a simulation-based operational replica of the UAV pavement-inspection process, designed to support controlled evaluation of inspection policies before field deployment. The DT models the states of the road network, the UAV, and dynamic traffic agents, which are continuously updated within the simulated environment to support online decision-making under occlusion and safety constraints. Unlike a static simulator, the proposed DT enables the evaluation and selection of recovery actions during mission execution. Formally, at discrete time t , the DT is represented as:

$$\mathcal{D}_t = (\mathcal{S}, \mathcal{A}_t, x_t, \mathcal{M}_t, \Pi), \quad (1)$$

where $\mathcal{S} = \{s_1, s_2, \dots, s_N\}$ denotes the set of road segments in the road network, $\mathcal{A}_t$ the set of dynamic agents, x_t the UAV state, $\mathcal{M}_t$ the inspection memory, and Π the set of candidate recovery policies. An example road segment s is shown in green in Figure 1d.

TABLE I: Adaptive recovery policies.

Policy	Action	Description
Hover	$p_{t+1} = p_t$	Remains stationary for a short duration Δt_{wait} to allow temporary occlusions to clear.
Micro	$p_{t+1} = p_t + \delta_t$	Performs a small displacement ($\ \delta_t\ \leq d_{\text{max}}$) to improve visibility while remaining close to the target segment.
Skip	Continue to next segment	Skips the occluded segment and revisits it later following the predefined inspection path.

The UAV state at time t is defined as:

$$x_t = (p_t, h_t, v_t, e_t), \quad (2)$$

where $p_t \in \mathbb{R}^2$ is the planar position, $h_t \in \mathbb{R}_{>0}$ is the altitude, $v_t \in \mathbb{R}_{\geq 0}$ is the speed, and $e_t \in [0, 1]$ is the battery level. Each dynamic agent $a_i \in \mathcal{A}_t$ is represented as:

$$a_i^t = (y_i^t, \nu_i^t, \kappa_i), \quad (3)$$

where y_i^t is the position, ν_i^t is the velocity, and $\kappa_i \in \{\text{vehicle}, \text{pedestrian}\}$ is the class label.

For each road segment s , we define a visibility score $o_{s,t} \in [0, 1]$, where $o_{s,t} = 1$ indicates that the segment is fully observable by the UAV at time t , and $o_{s,t} = 0$ indicates that it is completely occluded or outside the field of view. Intermediate values represent partial visibility due to dynamic occlusions. The inspection memory is then defined as:

$$\mathcal{M}_t = \{m_s^t \mid s \in \mathcal{S}\}, \quad (4)$$

where each segment state is given by:

$$m_s^t = \begin{cases} \text{inspected,} & \text{if } o_{s,t} \geq \tau_o, \\ \text{pending,} & \text{otherwise.} \end{cases} \quad (5)$$

When the visibility of the currently targeted segment s^* falls below a predefined threshold, i.e., $o_{s^*,t} < \tau_o$, the DT activates a recovery policy selected from Π . The Baseline strategy follows the planned inspection route without applying any recovery action when occlusions occur. The proposed framework considers three policies to handle occlusions: hover-and-recheck, micro-repositioning, and skip-and-revisit, as summarised in Table I. The waiting time Δt_{wait} represents a short duration during which the UAV remains stationary, while the displacement vector $\delta_t \in \mathbb{R}^2$ defines a local movement applied during the micro-repositioning strategy, constrained by $\|\delta_t\| \leq d_{\text{max}}$, where d_{max} is the maximum allowed displacement. Figure 3 illustrates the overall DT architecture.

IV. MULTITASK UAV PERCEPTION MODEL

The proposed framework embeds a perception layer within the UAV platform, which observes the environment from a top-down perspective. During UAV inspection, the detector processes RGB frames from the simulated camera to identify road-surface defects, pedestrians, and vehicles. Road-defect

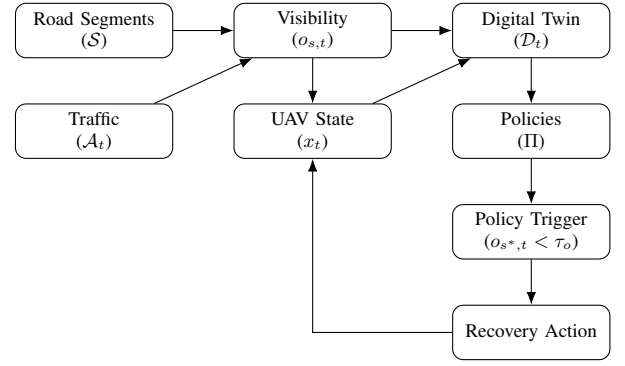


Fig. 3: Digital twin architecture for traffic-aware UAV pavement monitoring. The digital twin integrates road segments, traffic agents, UAV state, and visibility estimates to trigger recovery actions under dynamic occlusions.

detections support automatic image capture and dataset generation, while pedestrian and vehicle detections support traffic-aware inspection by identifying objects that may occlude the road surface. In this way, the detection module links the visual simulation environment with the DT decision layer described in Section III-C. The YOLO family of object-detection models was selected because of its balance between inference speed and detection accuracy.

A. Dataset Description and Preprocessing

A multi-source dataset was constructed from six road-damage datasets and two additional UAV datasets for traffic-agent classes. Since these sources differed in annotation format, image resolution, and original class taxonomy, they were normalised into a common three-class road-defect scheme consisting of *Single Crack*, *Crocodile Crack*, and *Pothole*. HighRPD [20] originally provided three classes: *line*, *block*, and *pit*. The *line* and *block* classes were remapped to *Single Crack* and *Crocodile Crack*, respectively, while images containing at least one *pit* annotation were discarded. The pothole-recognition dataset of Silva et al. [3] distinguished between *crack* and *pothole*, which were mapped directly to *Single Crack* and *Pothole*. PothRGBD [21] contained only pothole instances annotated as polygons; these annotations were converted to enclosing bounding boxes and retained under the *Pothole* class.

UAPD [22], [23] and UAV-PDD2023 [24] followed a common distress taxonomy including longitudinal, transverse, oblique, and alligator cracks, as well as *pothole* and the non-target *repair* class. For both, alligator-pattern cracks were mapped to *Crocodile Crack*, longitudinal, transverse, and oblique cracks were merged into *Single Crack*, and *repair* instances were discarded. Finally, the China UAV subset of RDD2022 [13] originally used D00, D10, D20, and D40 labels for longitudinal cracks, transverse cracks, alligator cracks, and potholes, respectively, together with *repair* and *block* labels. D00 and D10 were merged into *Single Crack*, D20 was

TABLE II: Summary of the datasets used to construct the final five-class dataset.

Source Dataset	Instances per class						Images		
	Single	Crocodile	Pothole	Person	Car	Total	Annotated	Background	Total
HighRPD [20]	11,409	6,900	0	0	0	18,309	9,974	310	10,284
Pothole-Recog. [3]	108	0	453	0	0	561	111	11	122
PothRGBD [21]	0	0	972	0	0	972	871	0	871
UAPD [22], [23]	3,256	0	94	0	0	3,350	2,146	0	2,146
UAV-PDD2023 [24]	10,074	603	195	0	0	10,872	2,403	0	2,403
RDD2022* [13]	2,689	293	86	0	0	3,068	1,919	482	2,401
UAV car detection [25]	0	0	0	0	16,868	16,868	299	0	299
Pedestrian recognition [26]	0	0	0	17,034	0	17,034	215	0	215
Merged Dataset	27,536	7,796	1,800	17,034	16,868	71,034	17,938	803	18,741
Balanced Dataset	24,295	24,295	24,295	24,000	23,884	120,769	42,755	3,420	46,175
Synthetic Dataset	4,435	3,134	4,665	8,376	5,333	25,943	2,235	0	2,235

*Only the China UAV subset of RDD2022 was considered.

mapped to *Crocodile Crack*, D40 was mapped to *Pothole*, and the *repair* and *block* labels were discarded.

After class-level normalisation, all annotations were converted into a common Pascal VOC/XML representation to enable consistent merging across sources. The unified dataset was then exported into the YOLO object-detection format for model training. Images were downsampled to a maximum side length of 640 px, with bounding-box coordinates updated accordingly. Images without valid annotations were discarded unless they were later used as verified background samples. Table II reports the resulting object-instance distribution. Because the merged dataset was strongly imbalanced across defect classes, an augmentation pipeline was applied using the Albumentations library [27]. The transformations included horizontal flipping, brightness and contrast adjustment, Gaussian noise, and rotations of up to $\pm 15^\circ$. Bounding boxes were updated during augmentation, and boxes with insufficient visible area after transformation were removed.

To make the detector suitable for traffic-aware inspection, two additional classes were incorporated: *Person* and *Car*. Object instances for both categories were extracted from public top-view UAV datasets [25], [26] and composited onto the merged road-damage images under controlled placement rules. A manually reviewed background set was also included to reduce false positives on undamaged road surfaces. The background images were verified to contain no visible road damage, people, or vehicles before being resized and augmented using a lighter transformation pipeline. As shown in Table II, the final *Balanced Dataset* contains 46,175 images and 120,769 annotated bounding boxes across the five classes. The *Balanced Dataset* was split into training, validation, and test subsets using a class-stratified 70%/20%/10% ratio, ensuring that the relative proportion of each class was preserved across all three subsets.

Finally, since the target domain is a Unity-based environment with synthetically generated road damage, vehicles, and pedestrians, we collected an additional *Synthetic Dataset*. This dataset consists of top-view UAV captures collected within the DT simulation framework, showing road scenes with instances

from the five classes.

B. Model Development and Training

To select the object detector with the best trade-off between accuracy, speed, and deployment cost for simulator integration, a preliminary first-stage detection experiment was conducted. YOLOv8 [28], YOLO11 [29], and YOLO12 [30] were compared across their nano, small, medium, and large variants using three coarse detection classes: *Road Defect*, *Person*, and *Car*. For this model-selection experiment, the road-defect subtypes *Single Crack*, *Crocodile Crack*, and *Pothole* were merged into a single *Road Defect* class, while *Person* and *Car* were retained as separate classes. All models were initialised from COCO pre-trained weights and evaluated on the same dataset split. The evaluation considered detection-accuracy and computational-efficiency metrics, including mean average precision (mAP) at an intersection-over-union (IoU) threshold of 0.5, mAP averaged over IoU thresholds from 0.5 to 0.95, precision, recall, model size, frames per second (FPS), and giga floating-point operations (GFLOPs), following the definitions in [31]. Based on the results reported in Table IV, YOLOv8n was selected as the most suitable trade-off between detection accuracy, model size, computational cost, and inference speed.

After selecting YOLOv8n as the base detector under the three-class configuration used during model selection, we implemented a modified YOLOv8n architecture capable of producing the final five-class output required by the simulator. In the first-stage detector, the road-defect categories were treated as a single *Road Defect* class, alongside *Person* and *Car*, preserving the same detection setup used for comparing candidate YOLO models. To recover the fine-grained road-defect labels, a lightweight classification head was then integrated using road-defect feature-pyramid embeddings to distinguish among *Single Crack*, *Crocodile Crack*, and *Pothole*. In this way, the final system retained the efficient detection capability of YOLOv8n while extending it to provide the five semantic classes required for simulator-based evaluation.

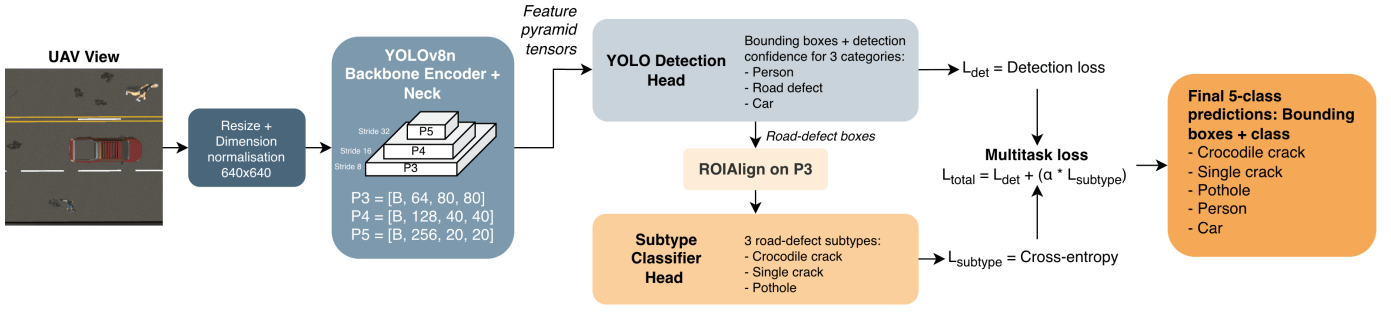


Fig. 4: Proposed multitask model based on YOLOv8n. B and P denote batch size and pyramid level, respectively.

The YOLOv8n encoder and both heads were first trained using the *Balanced Dataset*, detailed in Table II, which contains real-world image samples. To better adapt the model to the target domain, the trained shared-backbone YOLOv8n multitask model was used as the initialisation checkpoint and further optimised on the *Synthetic Dataset*. As illustrated in Figure 4, the multitask model first detects road defects and dynamic occluding agents, while the second head performs fine-grained classification only on regions identified as road damage. Thus, the YOLO backbone is used once to encode each image, and only embeddings corresponding to detected road defects are passed to the second head, enabling efficient road-distress classification.

In the implemented architecture, the YOLOv8n backbone and neck produce a feature pyramid composed of the P3, P4, and P5 tensors. These tensors are used by the YOLO detection head to predict coarse object categories: *Road Defect*, *Person*, and *Car*. For the subtype branch, only the P3 feature map is used, since it has the highest spatial resolution in the feature pyramid and is therefore better suited for representing small or thin road-surface defects. Road-defect bounding boxes are projected onto this P3 feature map and processed using ROIAlign, which extracts fixed-size feature representations for each road-defect region without cropping the original RGB image or executing a second backbone. These ROI-aligned features are then passed to the subtype classifier to distinguish *Crocodile Crack*, *Single Crack*, and *Pothole*. The multitask objective combines the YOLO detection loss with the subtype classification loss as

$$L_{\text{total}} = L_{\text{det}} + \alpha L_{\text{subtype}}, \quad (6)$$

where L_{det} is the standard YOLO detection loss, L_{subtype} is a cross-entropy loss over the three road-defect subtypes, and α controls the contribution of the subtype branch. In this work, α was set to 1.0, giving equal weight to the detection and subtype objectives.

V. EXPERIMENTAL RESULTS

This section reports the results of the DT recovery strategies and the UAV perception pipeline. The DT simulations were tested on a local desktop computer equipped with an AMD Ryzen 5 5600G processor at 3.90 GHz, an NVIDIA GeForce GTX 1050 Ti GPU with 4 GB of VRAM, 16 GB of RAM at

2666 MT/s, and a 64-bit x64 system architecture. The YOLO variants were trained under identical conditions, and FPS was measured on a single NVIDIA T4 GPU for consistency.

A. Digital Twin Recovery Strategies

Table III reports the inspection coverage achieved by the four navigation strategies under different traffic densities and flight altitudes. Values are presented as the mean $\pm$ standard deviation over 20 independent experimental repetitions. Additional operational metrics, including mission time, energy consumption, and recovery ratio, were also logged and are discussed where they affect the interpretation of each strategy. Under low-traffic conditions, Baseline achieved the highest coverage at medium and high altitudes, reaching $84.01 \pm 5.72\%$ and $82.82 \pm 6.79\%$, respectively. At low altitude, however, Micro produced the highest coverage ($60.30 \pm 11.81\%$). Although Skip enabled revisits to previously occluded regions, these additional manoeuvres did not translate into higher coverage in this configuration and instead increased mission duration and energy consumption.

TABLE III: Inspection coverage of the baseline and recovery strategies under different traffic densities and flight altitudes (mean $\pm$ standard deviation, %). Low, medium, and high altitudes correspond to 6 m, 10 m, and 15 m, respectively.

Traffic	Altitude	Baseline	Hover	Micro	Skip
Low	Low	56.00 ± 11.48	52.35 ± 11.89	60.30 ± 11.81	47.45 ± 11.78
Low	Medium	84.01 ± 5.72	83.59 ± 7.74	82.70 ± 10.47	79.63 ± 10.64
Low	High	82.82 ± 6.79	82.45 ± 10.29	76.41 ± 8.83	77.75 ± 8.12
Medium	Low	52.37 ± 10.18	53.90 ± 11.55	53.58 ± 9.26	51.71 ± 11.81
Medium	Medium	84.53 ± 8.15	80.43 ± 11.01	81.82 ± 7.35	81.03 ± 9.10
Medium	High	75.19 ± 11.42	81.98 ± 9.99	74.60 ± 10.70	75.03 ± 11.52
High	Low	50.78 ± 10.74	54.79 ± 14.08	51.80 ± 11.40	49.45 ± 11.22
High	Medium	81.87 ± 11.16	81.51 ± 7.98	79.13 ± 9.61	74.91 ± 12.30
High	High	78.28 ± 10.47	75.53 ± 9.85	73.01 ± 15.55	76.84 ± 11.03

Under medium-traffic conditions, Hover achieved the highest coverage at low and high altitudes ($53.90 \pm 11.55\%$ and $81.98 \pm 9.99\%$), whereas Baseline performed best at medium altitude ($84.53 \pm 8.15\%$). The differences between Hover and Baseline were relatively small, especially at medium and high altitudes, indicating that the benefit of recovery depends on the local traffic and flight configuration. Similarly, Skip continued to trigger revisit actions, but these actions mainly increased mission time and energy use without producing a substantial coverage gain. Micro remained broadly stable across the

TABLE IV: Comparison of YOLOv8, YOLO11, and YOLO12 variants for first-stage detection.

Metrics	YOLOv8				YOLO11				YOLO12			
	n	s	m	l	n	s	m	l	n	s	m	l
mAP@0.5	0.866	0.875	0.878	0.876	0.869	0.876	0.874	0.873	0.874	0.875	0.872	0.869
mAP@0.50:0.95	0.775	0.785	0.791	0.789	0.778	0.785	0.785	0.783	0.784	0.786	0.783	0.780
Precision	0.888	0.887	0.905	0.898	0.890	0.895	0.893	0.889	0.901	0.890	0.896	0.885
Recall	0.840	0.856	0.849	0.855	0.847	0.853	0.851	0.854	0.844	0.854	0.847	0.847
Size (MB)	6.0	21.5	49.6	83.6	5.2	18.3	38.6	48.8	5.3	18.1	38.9	51.0
FPS	273.47	97.73	39.19	22.68	263.11	92.38	35.98	32.36	166.43	63.46	26.03	18.18
GFLOPs	8.1	28.4	78.7	164.8	6.3	21.3	67.7	86.6	6.3	21.2	67.1	88.5

evaluated settings, although its coverage was generally lower than that of Baseline or Hover.

Under high-traffic conditions, low-altitude coverage decreased for all strategies because persistent vehicle occlusions reduced the visible road surface. Hover achieved the highest mean coverage at low altitude ($54.79 \pm 14.08\%$), but the large standard deviation indicates considerable run-to-run variability. In contrast, Baseline achieved the highest coverage at medium altitude ($81.87 \pm 11.16\%$) and high altitude ($78.28 \pm 10.47\%$). The clearest operational cost was observed for Hover under high traffic and low altitude, where the average mission time increased to 91.40 ± 118.27 s and energy consumption reached $5.08 \pm 6.57\%$. This occurred because Hover waits for the occluding vehicle to leave the inspection area before continuing; during congested runs, vehicles sometimes remained stationary for long periods, forcing the UAV to wait over the same location. Overall, the recovery strategies introduced trade-offs between coverage, recovery capability, mission duration, and energy consumption, but they did not consistently outperform Baseline in terms of coverage under the evaluated inspection configuration.

B. Multitask UAV Perception Model

Table IV presents the preliminary comparison of YOLOv8, YOLO11, and YOLO12 across their nano, small, medium, and large variants for first-stage detection, considering three coarse classes: *Road Defect*, *Person*, and *Car*. The results show that detection accuracy does not improve substantially when moving from nano to larger variants. For example, mAP@0.5 increases only from 0.866 for YOLOv8n to 0.878 for YOLOv8m, while inference speed drops from 273.47 FPS to 39.19 FPS and model size increases from 6.0 MB to 49.6 MB. A similar pattern is observed for YOLO11 and YOLO12. Therefore, YOLOv8n was selected due to its favourable balance between accuracy, training efficiency, model size, and real-time inference speed. These characteristics make it suitable for deployment inside the Unity simulator, where inference must run in real time.

The YOLOv8n-based multitask model described in Section IV-B was initially trained using the *Balanced Dataset* and subsequently fine-tuned with the *Synthetic Dataset*, as it represents the target domain. The first-stage detector was trained for 40 epochs, while the second-stage classifier was trained

for 10 epochs, both using early stopping and learning-rate scheduling. Evaluation was performed on the simulator test split and covered all five classes required by the DT. Before synthetic fine-tuning, the multitask YOLOv8n model achieved a five-class mAP@0.5 of 0.7442, mAP@0.50:0.95 of 0.6124, and macro F1-score of 0.7666. After fine-tuning, performance improved substantially, reaching 0.9591 mAP@0.5, 0.6450 mAP@0.50:0.95, and 0.9400 macro F1-score. The detection head also improved from 0.8525 to 0.9738 mAP@0.5, while the subtype branch remained highly reliable, reaching a macro F1-score of 0.9973 on matched predicted road-defect ROIs. The confusion matrix shown in Figure 5 further indicates that synthetic-domain fine-tuning improved localisation and detection coverage.

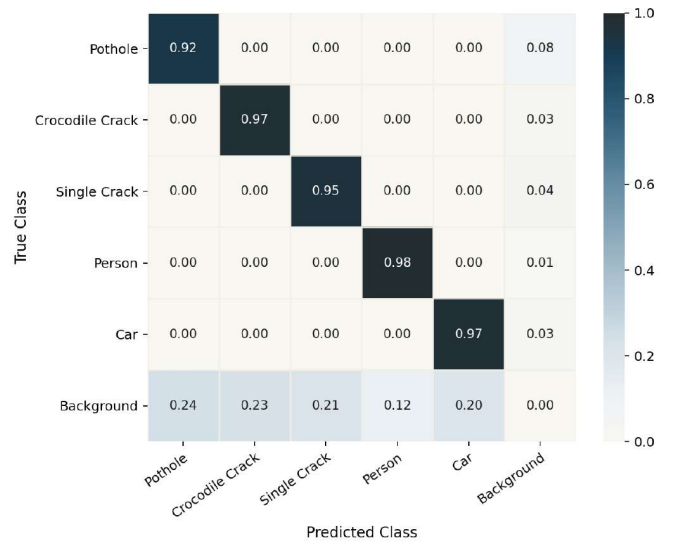


Fig. 5: Confusion matrix of the multitask detection model, fine-tuned and evaluated on the *Synthetic Dataset*.

VI. CONCLUSIONS

This paper presented a Unity-based digital twin framework for traffic-aware UAV pavement monitoring in open-traffic conditions. The framework integrates procedurally generated road defects, vehicles and pedestrians, UAV navigation, road-damage detection, and adaptive recovery strategies for occluded road regions. A multitask YOLOv8n perception model

was integrated into the simulator to detect road defects and traffic agents while classifying road-defect subtypes using ROI-aligned features from the same image encoding. After fine-tuning on the synthetic simulator dataset, the model achieved 0.9591 mAP@0.5, 0.6450 mAP@0.50:0.95, and 0.9400 macro F1-score for the final five-class output, indicating that synthetic-domain adaptation substantially improved the perception module used by the digital twin.

The simulation experiments showed that flight altitude had a strong effect on inspection coverage and that no single recovery strategy dominated across all traffic conditions. The evaluated policies produced different trade-offs: hover-and-recheck improved coverage in some medium- and high-traffic settings but could increase mission time under congestion, while skip-and-revisit produced non-zero recovery ratios at the cost of longer missions and higher energy consumption. Future work will focus on validating the framework with real UAV imagery, extending the traffic conditions considered in the simulator, incorporating weather effects, and developing adaptive policy-selection methods that choose recovery actions according to the current occlusion, altitude, and traffic state.

AUTHOR CONTRIBUTIONS

Conceptualization, Y.U. and E.S.; methodology, Y.U., G.L., E.S., and M.F.; software, Y.U., G.L., E.S., and M.F.; validation, Y.U. and E.S.; formal analysis, Y.U., E.S., and G.L.; investigation, Y.U., G.L., and E.S.; data curation, G.L.; writing—original draft preparation, Y.U., G.L., and E.S.; writing—review and editing, E.S.; visualization, Y.U., G.L., and E.S.; supervision, E.S. All authors have read and agreed to the published version of the manuscript.

REFERENCES

- [1] A. Dhiman and R. Klette, "Pothole detection using computer vision and learning," *IEEE Transactions on Intelligent Transportation Systems*, vol. 21, no. 8, pp. 3536–3550, 2019.
- [2] E. Salcedo, M. Jaber, and J. Requena Carrión, "A novel road maintenance prioritisation system based on computer vision and crowdsourced reporting," *Journal of Sensor and Actuator Networks*, vol. 11, no. 1, 2022. [Online]. Available: <https://www.mdpi.com/2224-2708/11/1/15>
- [3] L. A. Silva, H. Sanchez San Blas, D. Peral García, A. Sales Mendes, and G. Villarubia González, "An architectural multi-agent system for a pavement monitoring system with pothole recognition in uav images," *Sensors*, vol. 20, no. 21, 2020.
- [4] R. Zhao, Y. Huang, H. Luo, X. Huang, and Y. Zheng, "A framework for using UAVs to detect pavement damage based on optimal path planning and image splicing," *Sustainability*, vol. 15, no. 3, p. 2182, 2023.
- [5] P. J. López-González, D. Reyes-González, O. Moreno-Vázquez *et al.*, "Inspection and evaluation of urban pavement deterioration using drones: Review of methods, challenges, and future trends," *Future Transportation*, vol. 6, no. 1, p. 10, 2026.
- [6] C. Sierra, S. Paul, A. Rahman, and A. Kulkarni, "Development of a cognitive digital twin for pavement infrastructure health monitoring," *Infrastructures*, vol. 7, no. 9, p. 113, 2022.
- [7] M. A. Talaghat, A. Golroo, A. Kharbouch, M. Rasti, R. Heikkilä, and R. Jurva, "Digital twin technology for road pavement," *Automation in Construction*, vol. 168, p. 105826, 2024.
- [8] A. Consilvio, J. S. Hernández, W. Chen, I. Brilakis, L. Bartoccini, F. Di Gennaro, and M. van Welie, "Towards a digital twin-based intelligent decision support for road maintenance," *Transportation Research Procedia*, vol. 69, pp. 791–798, 2023.
- [9] M. M. Topu, M. A. Anik, A. T. Wasi, and M. M. Ahsan, "Digital twin-driven pavement health monitoring and maintenance optimization using graph neural networks," *arXiv preprint arXiv:2511.02957*, 2025.
- [10] Y. Zhong, S. Ye, Y. Liu, and J. Li, "A route planning method for UAV swarm inspection of roads fusing distributed droneport site selection," *Sensors*, vol. 23, no. 20, p. 8479, 2023.
- [11] I. S. A. Aburqaq, S. Naimi, S. Saedi, and M. A. A. Shahin, "Assessment of uav usage for flexible pavement inspection using gcps: Case study on palestinian urban road," *Sustainability*, vol. 17, no. 18, p. 8129, 2025.
- [12] O. Alzamzami, A. Babour, W. Baalawi, and L. Al Khuzayem, "Pds-uav: A deep learning-based pothole detection system using unmanned aerial vehicle images," *Sustainability*, vol. 16, no. 21, 2024. [Online]. Available: <https://www.mdpi.com/2071-1050/16/21/9168>
- [13] D. Arya, H. Maeda, S. K. Ghosh, D. Toshniwal, and Y. Sekimoto, "Rdd2022: A multi-national image dataset for automatic road damage detection," *Geoscience Data Journal*, vol. 11, no. 4, pp. 846–862, 2024. [Online]. Available: <https://rmdts.onlinelibrary.wiley.com/doi/abs/10.1002/gdj3.260>
- [14] J. Zeng and H. Zhong, "Yolov8-pd: an improved road damage detection algorithm based on yolov8n model," *Scientific reports*, vol. 14, no. 1, p. 12052, 2024.
- [15] S. Youwai, A. Chaiphaphat, and P. Chaipetch, "YOLO9tr: A lightweight model for pavement damage detection utilizing a generalized efficient layer aggregation network and attention mechanism," *Journal of Real-Time Image Processing*, vol. 21, p. 138, 2024.
- [16] Y. Hu, N. Chen, Y. Hou, X. Lin, B. Jing, and P. Liu, "Lightweight deep learning for real-time road distress detection on mobile devices," *Nature Communications*, vol. 16, p. 4212, 2025.
- [17] J.-C. Tsai, K.-T. Lai, T.-C. Dai, J.-J. Su, C.-Y. Siao, and Y.-C. Hsu, "Learning pothole detection in virtual environment," in *2020 International Automatic Control Conference (CACS)*, 2020, pp. 1–5.
- [18] W. Wang, X. Xu, J. Peng, W. Hu, and D. Wu, "Fine-grained detection of pavement distress based on integrated data using digital twin," *Applied Sciences*, vol. 13, no. 7, p. 4549, 2023.
- [19] S. Wang, B. Cai, W. Wang, Z. Li, W. Hu, B. Yan, and X. Liu, "Automated detection of pavement distress based on enhanced yolov8 and synthetic data with textured background modeling," *Transportation Geotechnics*, vol. 48, p. 101304, 2024.
- [20] J. He, L. Gong, C. Xu, P. Wang, Y. Zhang, O. Zheng, G. Su, Y. Yang, J. Hu, and Y. Sun, "HighRPD: High-resolution road pavement distress dataset," <https://data.mendeley.com/datasets/sywsj7djj/1>, 2024, mendeley Data, Version 1. Accessed: 30 September 2025.
- [21] M. Yurdakul and Şakir Tasdemir, "An enhanced yolov8 model for real-time and accurate pothole detection and measurement," 2025. [Online]. Available: <https://arxiv.org/abs/2505.04207>
- [22] J. Zhu, J. Zhong, T. Ma, X. Huang, W. Zhang, and Y. Zhou, "Pavement distress detection using convolutional neural networks with images captured via uav," *Automation in Construction*, vol. 133, 2022.
- [23] J. Zhong, J. Zhu, J. Huyan, T. Ma, and W. Zhang, "Multi-scale feature fusion network for pixel-level pavement distress detection," *Automation in Construction*, vol. 141, 2022.
- [24] H. Yan and J. Zhang, "UAV-PDD2023: A benchmark dataset for pavement distress detection based on UAV images," *Data in Brief*, vol. 51, p. 109773, 2023.
- [25] txt2csv, "UAV car detection dataset," Jul. 2022. [Online]. Available: <https://universe.roboflow.com/txt2csv/uav-car-detection-ksrly>
- [26] Ambulances, "Pedestrian recognition dataset," Nov. 2024. [Online]. Available: <https://universe.roboflow.com/ambulances/pedestrian-recognition-c3bb4>
- [27] A. Buslaev, V. I. Iglovikov, E. Khvedchenya, A. Parinov, M. Druzhinin, and A. A. Kalinin, "Albumentations: Fast and flexible image augmentations," *Information*, vol. 11, no. 2, p. 125, 2020.
- [28] Ultralytics, "Yolov8," <https://github.com/ultralytics/ultralytics>, 2023.
- [29] Ultralytics, "YOLO11," <https://docs.ultralytics.com/models/yolo11/>, 2024, accessed: 28 June 2026.
- [30] Y. Tian, Q. Ye, and D. Doermann, "Yolov12: Attention-centric real-time object detectors. arxiv 2025," *arXiv preprint arXiv:2502.12524*, 2025.
- [31] E. Salcedo, Y. Uchani, M. Mamani, and M. Fernandez, "Towards continuous floating invasive plant removal using unmanned surface vehicles and computer vision," *IEEE Access*, vol. 12, pp. 6649–6662, 2024.